hat a significant amount of time must be set aside to complete this process.

Long-term Support and Maintenance Costs

Backports can be an effective way to address security flaws and vulnerabilities in older versions of software. However, each backport introduces a fair amount of complexity within the system architecture and can be costly to maintain.

For example, Python 2.7.18 was the final official release of Python 2. In order to remain current with security patches and continue enjoying all of the new developments Python has to offer, organizations needed to upgrade to Python 3 or start freezing requirements and commit to legacy long-term support.

Backporting Side Effects

Each backport can create many unwanted side effects within the application environment. Just as an upstream software application affects all downstream applications, so too does a backport applied to the core software.

Additional Security Implications and Vulnerabilities

Backporting is a common technique to address a known bug within the IT environment. At the same time, relying on a legacy codebase introduces other potentially significant security implications for organizations. Relying on old or legacy code could result in introducing weaknesses or vulnerabilities in your environment. These issues affect not only the main application but also all dependent libraries and forked applications to public repositories. It is important to consider how each backport fits within the organization's overall security strategy, as well as the IT architecture.

SECTION: engineering/releases/internal-releases/index.md

Internal release overview

Internal Releases are private releases of GitLab for our single-tenant SaaS instances. They allow us to remediate
high-severity issues on Dedicated instances:

As quickly and efficiently as on GitLab.com
(SLA driven)
Without version skips in the public packages
Without disclosing vulnerabilities before a public patch release

Internal releases are performed according to a specific criteria:

Addressed critical (S1)
fixes (bug or security vulnerability) that impact GitLab Dedicated availability: Security or bug fixes
1. Security vulnerability: The SIRT team investigates a vulnerability and deems the issue to

be of high
severity.
2. Critical bug: The Dedicated team reports a high-severity issue causing a performance degradation.
Target the current minus one (N-1) and current minus two (N-2) GitLab versions
Deliver fixes through a private channel before public disclosure

If you·re a GitLab engineer looking to fix a high-severity via an internal release, please follow the steps on the
internal release runbook for GitLab engineers.

Internal release process

The end-to-end internal release process consists of the following stages:

Diagram source - internal

An internal release has the following phases:

1. Detect: A high-severity issue (S1) impacting GitLab Dedicated availability is identified.
Security vulnerability: The SIRT team investigates the vulnerability and deems the issue to be of a high
severity.
Critical bug: The Dedicated team reports a high-severity issue causing a performance degradation.

2. Prepare: The first step in the internal release process, when a release issue is created and stakeholders,
including the GitLab Dedicated Group are notified.

3. GitLab.com remediation:
The group relevant to the vulnerability/bug prepares the security fix on the GitLab security repositories.
Release managers merge the fix to the GitLab default branch.
The high-severity fix is deployed to GitLab multi-tenant production environment (GitLab.com).
In case of a vulnerability, the AppSec team
verifies that the vulnerability/bug has been remediated on GitLab.com.

4. Backports: Security merge requests targeting N-1 and N-2 stable branches are prepared by the relevant group
associated with the vulnerability/bug.
Engineers ensure the backports are ready to be merged (approvals, green pipelines, etc).
Once the backports are ready to be merged, release managers merge them into the stable branches.

5. Release: The internal CNG images and Omnibus packages are built and uploaded to the pre-release channel.

6. Final Steps: Roll out the internal release packages to GitLab single-tenant SaaS instances.
   The GitLab Dedicated Group is notified.
   A Dedicated emergency maintenance process
    starts.

SECTION: engineering/releases/patch-releases/index.md

Patch release overview

Patch releases are performed according to the GitLab Maintenance Policy:

 Backport bug fixes to the current stable released version of GitLab.
 Backport security fixes to the current, and previous two GitLab Versions.

Patches that are outside of our maintenance policy must be requested and agreed upon by the release managers and the requester (see backporting to versions outside the maintenance policy for details).

Patch releases are prepared in parallel with regular GitLab.com deployments so that continuous deployment is not blocked. In this way we can apply security fixes to GitLab.com instances before the public release.

If you're a GitLab engineer looking to:

 Include a security fix in a patch release, please follow the steps on the security runbook for GitLab engineers.
 Include a bug fix in a patch release, please follow the steps on the patch release runbook for GitLab engineers. Security vulnerabilities in GitLab and its dependencies are to be addressed following the Security Remediation SLAs.

Bug fixes are worked on in the GitLab canonical repositories, while security fixes are worked on in the mirrored GitLab security repositories
to avoid revealing vulnerabilities before the release.

 Patch release types

Patch releases have multiple touchpoints between many teams to prepare, validate and publish packages containing
bug and vulnerability fixes.

At GitLab, there are two types of patch releases processes:

1. Planned (default): An SLO-driven patch to publish all available bug and vulnerability fixes per
   the GitLab maintenance policy. Scheduled twice a month on
   the Wednesday before and after the monthly release week, planned patches comply
   with the bug SLO and
   the security remediation SLAs. Patches that include

critical vulnerabilities will be considered critical patches.

1. Unplanned: An immediate patch required outside of the planned patch release cadence to mitigate a high-severity (critical) vulnerability. These ad-hoc patches are the result of an incident, they require a security RCA done by the team that introduced the high-severity vulnerability and forced an unplanned release. The AppSec team is responsible for assessing the vulnerability, working with engineering to decide on the best approach to resolve it and coordinating with release managers on a timeline for the release.

Patch release process

The process for a patch release is the same for all types with one important distinction: planned patches include all bug and security fixes ready at the time of the patch release preparation, while unplanned patches will likely only include the security fix for high-severity vulnerability.

The end-to-end patch release process consists of the following stages:

Diagram source - internal

At any given time, GitLab Engineers prepare bug fixes for the current version and vulnerability fixes to the current and previous two GitLab versions:

Step 1a: Bug fix prepared - A merge request backporting a bug fix to the current version is prepared by GitLab engineers:

The merge request executes end-to-end tests via test-on-omnibus pipeline to guarantee the bug fix meets the quality standards.

If the test-on-omnibus pipeline fails, a review from a Software Engineer in Test is required.

The merge request is merged by a GitLab maintainer in the stable branch associated

Step 1b: Vulnerability fix prepared - Engineers fix vulnerabilities in the relevant Security repository. A fix is considered complete only when it has a security implementation issue with the following:

All checkboxes checked to show all steps have been completed.

An AppSec and Maintainer approved MR targeting the default branch.

A backport MR for each intended version. In most cases this will mean 4 MRs to cover each supported version. Each MR must have passing pipelines, required approvals and be assigned to the release bot for processing.

The ~"security-target" label is applied. This will automatically review the issue and link it to the security tracking issue if it is ready.

Two days before the planned due date, release managers start the patch release process, they make sure that all prepared bug and security fixes are safely released. Deployments to GitLab.com run in parallel.

A patch release has the following phases:

Step 2: First steps Release preparation begins when release managers run the prepare chatops command to create the new release task issue to guide the patch release. From here they follow the checklist to complete the initial set up and communication issues needed to prepare the release.
Step 3: Early Merge Phase - Release Managers deploy security fixes to GitLab.com. Fixes with the ~"security-target" label that are linked to the security tracking issue will have the MR targeting the default branch merged. This allows fixes to be deployed to GitLab.com before they are released to self-managed users.
Step 4: Merge backports - The day before the release due date, backports with security fixes targeting the supported versions are merged. At this point, everything included in the patch must be deployed to GitLab.com, and backports must apply to all stable branches.
Step 5: Release preparation - When all fixes are deployed and merge, Release managers prepare and test the packages.
Step 5a: Tag - Release managers tag new patch release packages for the current and previous two GitLab versions.
Step 5b: Deploy - The patch release package is deployed and tested to the GitLab release instance.
Step 5c: Release - Release managers publish the packages associated with the patch release.
Step 6: Final steps - At this point patch release packages are available to all users. Release managers wrap up the final steps of the patch release.
Step 6a: Patch release blog post is published
Step 6b: Default branches, stable branches, and tags are synced from Security to Canonical to return to our default state of working in the open.

Patch release FAQs

How can I backport a bug fix to the next patch release?

If you're a GitLab engineer looking to include a bug fix in a patch release, please follow the steps on the patch release runbook for GitLab engineers.

Where can I find the next patch release information?

Patch release information, including targeted versions, scheduled date and status can be found on the internal Grafana release dashboard

A security issue was assigned to me, where should I start?

See the Security process as Engineer documentation for more information.

Why wasn't my security fix included in the Patch Release?

Security issues created on GitLab Security need to be associated with the Security Tracking issue for them to be included on the Security
Release. Make sure to use the security issue template and follow the listed steps.

How many backports do I need when working on a security issue?

Besides the merge request targeting master, three backports will be needed targeting the last two monthly releases and the current release.
For more information, see security backports.

How can I revert a security merge request?

Once a security merge request has been merged, it's not advisable to revert it for multiple reasons:

1. Reverting requires rolling back a security fix, compromising the integrity of GitLab.com and self-managed instances
1. Reverting without making another fix into the release risks disclosing the vulnerability to the public when we make the release.
   This is unacceptable
1. Patch releases are performed in a restricted time constraint, reverting a security merge request requires another fix to be
   prepared in time to avoid impacting the patch release timeframe

If a security vulnerability introduced a non-vulnerability bug, in most cases, the appropriate path forward is to fix the issue in the canonical repository (after the patch release has been published).

If a security vulnerability introduced a high severity non-vulnerability bug, engage with AppSec and release managers to coordinate next steps.

For more information, see How to Mitigate Bugs Introduced by Security Merge Request runbook.

SECTION: engineering/testing/_index.md

Welcome to the Testing Guide. Pages in this section provides information about testing practices, methodologies, and tools used in our development workflow. Effective testing is crucial for maintaining code quality, preventing regressions, and ensuring that our software meets requirements.

Introduction to Testing at GitLab

This introduction provides new engineers with an
overview of our testing philosophy, practices, and the support available to help you contribute effectively to our quality engineering efforts.

How We Test

Our Testing Philosophy

At GitLab, we believe that quality is everyone's responsibility, and testing is integrated into every stage of our development process rather than being a separate phase. Our approach is built on industry best practices and GitLab's core values.

Test Pyramid Approach: We champion the concept of the test pyramid, prioritizing fast, reliable tests at the base (unit tests) while using fewer, more focused tests at higher levels (integration and end-to-end). This approach gives us:

- Rapid feedback during development
- Reliable detection of regressions
- Efficient use of CI/CD resources
- Maintainable test suites

Strategic Testing Focus: Our testing strives to consider risk analysis and input on test strategy, helping us focus testing efforts on critical user journeys and high-impact areas. We make strategic decisions about where to invest our testing efforts based on user impact and business needs.

Quality Gates: Testing is embedded throughout our product development workflow:

- Pre-commit and pre-receive hooks for immediate feedback
- Merge request pipelines with mandatory code reviews that must pass before code integration
- Deployment pipelines with comprehensive test suites
- Post-deployment monitoring and validation

Testing Ownership Model

Everyone Tests: While we have dedicated the Test Governance team and greater Developer Experience department, every product engineering team is responsible for creating and maintaining tests for their features. Developer Experience are there for supporting engineers in creating comprehensive test coverage that includes:

- Writing unit tests for new functionality
- Adding integration tests for API endpoints and service interactions
- Contributing to end-to-end test coverage for critical user flows
- Maintaining and fixing flaky or outdated tests

Test Governance Support: The Test Governance team provides:

- Testing infrastructure and tooling through Developer Experience teams
- Collaboration with Developer Experience for development workflow support
- Guidance on testing strategies and best practices
- Support for complex testing scenarios
- Test automation frameworks and libraries

When We Test

Development Workflow Integration

Testing Early and Often: We encourage writing tests alongside feature development to ensure clear requirements understanding, better code design, and comprehensive coverage from the start.

Continuous Integration: Every merge request triggers automated testing:

- Unit and integration tests run on every push
- Feature tests execute for UI changes
- Performance tests validate critical paths
- Security scans check for vulnerabilities
- End-to-end tests for critical user journeys

Release and Deployment Testing

Pre-Deployment Validation: Before code reaches production:

- Smoke tests verify basic functionality with staging-canary blocking deployments
- Performance tests ensure acceptable response times
- End-to-end tests validate critical user journeys
- Canary deployments allow gradual rollout with monitoring

Post-Deployment Monitoring: Testing doesn't stop at deployment. Post-deployment monitoring is also done including:

- Synthetic monitoring simulating user interactions
- Performance monitoring tracking application health
- Error tracking identifying issues in real-time
- Feature flag testing enabling safe experimentation

Support Available

Getting Help with Testing

Request for Help Process: When you need testing support or guidance, use our established request process:

- Create an issue using the testing support template
- Include context about your testing challenges or requirements
- Expect response within our defined SLA timeframes

Developer Experience Department

Developer Experience Team: Provides testing infrastructure, tools, and frameworks including:

- Testing pipeline optimization
- Test automation libraries and utilities
- CI/CD testing infrastructure
- Performance testing capabilities

Test Governance Team: Support specific product areas with:

- Testing strategy guidance
- Complex test scenario design
- Flaky test investigation and resolution

- Test coverage analysis and recommendations

On-Call Support: Engineering teams participate in incident management rotations to ensure rapid response to production issues

Self-Service Resources

Documentation and Guides

- GitLab Testing Guide - Guidelines for automated testing in the GitLab project
- Testing Levels, Tooling, and Strategy -
  Detailed technical implementation guide
- Testing Best Practices - Everything you should know about how to write good tests in the GitLab project
- Code Review Guidelines - Mandatory review process for all merge requests
- Product Engineer guide to E2E test failure issues

· GitLab End-to-End Testing Overview (Video)

<figure class="videocontainer">
  <iframe src="https://www.youtube.com/embed/KbQzrVJMvNQ" frameborder="0" allowfullscreen="true"> </iframe>
</figure>

Duration: ~30 minutes
Level: Beginner to Intermediate

This video covers:

- · Directory structure and test organization
- ·· Finding and understanding existing tests
- · Deep dive into E2E test architecture
- · Where E2E tests run in our infrastructure
- · Debugging test failures from merge requests
- · Working with feature flags in tests
- · Troubleshooting common failure issues
- · Running tests locally in your GDK

- Presentation Slides

Further Reading

- The Practical Test Pyramid - A deep dive into the "Test Pyramid"

Community and Communication

- Testing-focused Slack channels for questions and discussions are namely gtestgovernance and the broader sdeveloperexperience

Tooling and Automation

- Test generators and templates for common scenarios
- Automated workflow tooling for issue and MR triage
- CI/CD pipeline templates with testing best practices
- Performance and coverage monitoring dashboards

For detailed technical implementation guidance, refer to our comprehensive Development Testing Guide.
For immediate testing support, use our request for help process.

SECTION: engineering/testing/browser-performance-testing.md

The GitLab Browser Performance Tool (GBPT) is a SiteSpeed wrapper that measures frontend performance in browsers, providing insights into web page performance across GitLab environments. The tool is maintained by the Performance Enablement team.

Overview

GBPT specifically focuses on web page frontend performance metrics in browsers. The tool runs against reference architecture environments to ensure consistent and reliable frontend performance measurements.

Testing Process

Test Environments

GBPT testing currently runs against the 1k Reference Architecture test environment, maintained by GitLab Delivery: Framework and the Staging environment daily.

Results and Analysis

Test results are maintained in the GBPT Performance Wiki, including:

- Frontend performance metrics
- Page load timing data
- Browser-specific measurements

SECTION: engineering/testing/distribution.md

Overview

The goal of this page is to document existing Quality Engineering activities in Distribution group.

Dashboards

- QE Distribution dashboard - Dashboard to track Quality Engineering work items
- Distribution Issues - Dashboard to visualize metrics important for Bug Prioritization
- Bug Prioritization metrics - Bugs metrics required for Bug Prioritization (ensure to filter by Distribution group)

Quality work

Quality work is being tracked in epic9057. The epic lists large initiatives that need to be worked on to better support quality in Distribution group.

GitLab QA

GitLab QA is being used in several Distribution projects to validate that GitLab works as expected.

Project	Tests type	Schedule
GitLab Omnibus	Full	QA mirror pipeline is triggered
GitLab Charts	Sanity	Run automatically in merge requests and scheduled against default branch
GitLab Charts	Full	Triggered manually in merge requests
GitLab Operator	Smoke	Run automatically in merge requests
GitLab Operator	Full	Manually triggered
Reference Architecture Tester	Full	Manually triggered and FIPS QA Nightly

Check Running GitLab QA for information on how to run GitLab QA locally for development.

Investigate QA failures

1. Search for the failure in open Pipeline Triage issue or search for the spec name in the main GitLab project
    - If Allure report is available: Click on report link -> Product defects -> Select failed spec -> click Failure issues. Demo
    - Some specs might have multiple QA failure issues with different stack trace. In such case, compare failed stack trace from the job with the ones listed in the issues.
1. If an issue with the same error is not found
    - Continue to debug the QA failure following the guide
    - Reach out to the Developer Experience stage - on-call DRI or Distribution SET

Bug Prioritization

The Distribution team works together on Bug Prioritization and aims to close at least 6 bugs per milestone based on the team's current availability. The number of bugs per milestone is revisited in the following issue1100.

Process:

- Team creates a new Planning issue
- SET creates a new issue using Bug Prioritization template
- SET reviews open bugs using Distribution Issues
  - Add Severity labels to bugs that are missing a severity label
  - Review open bugs following Prioritization Guidelines
- SET to propose in team planning issue 6 bugs to be considered in milestone
- At the end of the quarter:
  - SET reviews Distribution Issues metrics for open bugs
  - SET shares analysis with the Distribution team
  - The Distribution discusses if the process should be adjusted

Quad Planning

Quality team reviews open issues for quad planning following Quad Planning process.

SECTION: engineering/testing/end-to-end-pipeline-monitoring.md

End-to-end (E2E) test pipelines

The test pipelines run on a scheduled basis, and their results are posted to Slack. The following are the end-to-end test pipelines that are monitored every day.

| Environment | Links
| Tests type | Frequency
| Slack channel
| Latest test report
|
|---|---

---|---|---

---|
| Production | Pipelines \| Definition
| Smoke | after each deployment to Canary
| e2e-run-production |
Production Sanity |
| Canary | Pipelines \| Definition \| Chatops |
Smoke | after each deployment to Canary and after a feature flag in production has been updated to true or 100% | e2e-run-production
| Canary Sanity |
| Staging | Pipelines \| Definition
| Smoke | after each deployment to Staging-Canary

e2e-run-staging	Staging Sanity
-	Pipelines | Definition	Smoke	after the execution of post-deploy migrations in Staging	e2e-run-staging	Staging Sanity
Staging Canary	Pipelines | Definition	Smoke	after each deployment to Staging-Canary	e2e-run-staging	Staging-Canary Sanity
-	Pipelines | Definition | Chatops	Full	after each deployment to Staging-Canary and after a feature flag in Staging has been updated to true or 100%	e2e-run-staging	Staging-Canary Full
CustomersDot Staging	Pipelines | Definition	Full	after each deployment to CustomersDot Staging	e2e-run-staging sfulfillmentstatus	CustomersDot Staging
CustomersDot Production	Pipelines | Definition	Smoke	after each deployment to CustomersDot Production	e2e-run-production sfulfillmentstatus	N/A
Preprod	Pipelines | Definition	Smoke	Every month for a few days before release, at 03:00 UTC and after deployment to preprod during Security and Patch releases	e2e-run-preprod	Preprod
Release	Pipelines | Definition	Smoke	Every month after the final release and after deployment to Release during Security and Patch releases	e2e-run-release	Release
GitLab master e2e:test-on-omnibus-ee	Pipelines | Definition	Full	scheduled pipeline every 2 hours	e2e-run-master	Master EE
GitLab master e2e:test-on-omnibus-ce	Pipelines | Definition	Full	Daily at 4:00am UTC	e2e-run-master	Master CE
GitLab master e2e:test-on-gdk	Pipelines | Definition	Full	scheduled pipeline every 2 hours	e2e-run-master	Master GDK
GitLab master e2e:test-on-cng	Pipelines | Definition	Smoke	scheduled pipeline every 2 hours	e2e-run-master	Master CNG
GitLab master Nightly	Pipelines | Definition	Full	Daily at 4:00am UTC	e2e-run-master	Master Nightly

NOTE:
For information on how to investigate failing end-to-end tests and pipelines, check out Debugging Failing Tests and Test Pipelines

Visual Pipeline Environment Map

This diagram provides a visual representation of how our end-to-end test pipelines map to various environments in our infrastructure. It illustrates the flow of tests across Merge Requests, Development, Staging, Preprod, and Production environments, showing when tests are triggered (on commit, after deploy, after configuration changes, etc.) and what type of tests run in each environment (smoke, full, etc.).

The color coding indicates test types and environment categories, making it easier to understand our comprehensive testing strategy across the entire deployment pipeline. This visualization is particularly valuable for infrastructure planning. The original LucidChart diagram also includes a Cells version so we can visualize what it will look like.

Test metrics

For visibility on the test health, we have test execution results exported to:

- an InfluxDb instance and use Grafana dashboards to visualize the results
- a Google Cloud Storage (GCS) instance which is then accessible through Snowflake

Test reports

Allure report

Another tool we have to present the test results is through the Allure test reports. Tests that run on pipelines generate Allure reports. The QA framework uses the Allure RSpec gem to generate source files for the Allure test report. An additional job in the pipeline:

- Fetches these source files from all test jobs.
- Generates and uploads the report to the S3 bucket gitlab-qa-allure-report located in AWS group project eng-quality-ops-ci-cd-shared-infra.

Each type of scheduled pipeline generates a static link for the latest test report according to its stage:

Environment	Description	Link
master (gdk)	E2E test execution against gitlab-development-kit environment packaged in a Docker container.	{{< engineering/test-execution-allure-link "e2e-test-on-gdk" >}}
master (test-on-omnibus)	E2E test execution against various configurations of omnibus

images. |
{{< engineering/test-execution-allure-link "e2e-test-on-omnibus" >}} |
| nightly | E2E test execution against various configurations of omnibus nightly images. |
{{< engineering/test-execution-allure-link "nightly" >}} |
| staging-full | E2E test execution against https://staging.gitlab.com environment.
| {{< engineering/test-execution-allure-link "staging-full" >}} |
| staging-sanity | E2E test execution against various configurations of omnibus nightly images. |
{{< engineering/test-execution-allure-link "staging-sanity" >}} |
| staging-ref-full | E2E test execution against https://staging-ref.gitlab.com environment. |
{{< engineering/test-execution-allure-link "staging-ref-full" >}} |
| staging-ref-sanity | E2E test execution against https://staging-ref.gitlab.com environment. |
{{< engineering/test-execution-allure-link "staging-ref-sanity" >}} |
| preprod | E2E test execution against https://pre.gitlab.com environment.
| {{< engineering/test-execution-allure-link "preprod-sanity" >}} |
| production-full | E2E test execution against https://gitlab.com environment.
| {{< engineering/test-execution-allure-link "production-full" >}} |
| production-sanity | E2E test execution against https://gitlab.com environment.
| {{< engineering/test-execution-allure-link "production-sanity" >}} |

These reports are also included in the pipeline status alerts on Slack.

Test session issue

For each end-to-end pipeline that runs in the various environments we automatically test, we create a test session issue that contains the test session information. Test session issues group test results by DevOps stages, and link to test cases, and test failure issues.

Example of a test session issue: <https://gitlab.com/gitlab-org/quality/testcase-sessions/-/issues/72516>

Test session issues are a workaround for a missing GitLab feature. Once GitLab stores test data, we can improve failure reporting and management.

Test result issue

Each test is associated to a GitLab testcase.

```
ruby
  RSpec.describe 'Stage' do
    describe 'General description of the feature under test' do
      it 'test name', testcase: 'https://gitlab.com/gitlab-org/gitlab/-/quality/testcases/:testcaseid' do
        ...
      end
```

```
    it 'another test', testcase: 'https://gitlab.com/gitlab-
org/gitlab/-/quality/testcases/:anothertestcaseid' do
      ...
    end
  end
end
```

The test failure stack trace and the issue stack trace are compared, and the existing issue for which the stack trace is the most similar (under a 15% difference threshold) to the test failure is used. The test failure job is then added to the failure report list in the issue. Group label is automatically inferred based on the productgroup metadata of the test.

SECTION: engineering/testing/flaky-tests.md

Introduction

A flaky test is an unreliable test that occasionally fails but passes eventually if you retry it enough times. Flaky tests can be a result of brittle tests, unstable test infrastructure, or an unstable application. We should try to identify the cause and remove the instability to improve quality and build trust in test results.

Manual flow to detect flaky tests

When a flaky test fails in an MR, the author might follow the following flow:

mermaid
graph LR
  A[Test fails in a MR] --> C{Does the failure looks related to the MR?}
  C -->|Yes| D[Try to reproduce and fix the test locally]
  C -->|No| E{Does a flaky test issue exists?}
  E -->|Yes| F[Retry the job and hope that it will pass this time]
  E -->|No| G[Wonder if this is flaky and retry the job]

Why is flaky tests management important?

- Flaky tests undermine test results, leading to engineers disregarding test failures as flaky.
- Manual retries to try to get flaky tests to pass, and the effort needed to investigate flaky tests as failures are a significant waste of time.
- Managing flaky tests by quickly fixing the cause or removing the test from the test suite allows test time and costs to be used where they add value.

Flaky tests management process

We started an experiment to automatically open merge requests for very flaky tests to improve overall pipeline stability and duration.
To ensure that our product quality is not negatively affected due to test coverage reduction,

the following process should be followed:

1. Groups are responsible for reviewing their test-quarantining merge requests.
   These merge requests are meant to start a discussion on whether a test is useful or not.
   In case a test is impacting master' stability heavily, the Engineering Productivity team can merge these merge requests even without a review from their responsible group.
   The group should still review the merge request and start a discussion about the quarantined test's next step.
2. Once a test is quarantined, its associated issue will be reported in weekly group reports.
    Groups can also list all of their flaky tests and their quarantined tests (replace group::xxx in the issues list).
3. The number of quarantined test cases per group is also available as a dashboard.
4. Groups are responsible for ensuring stability and coverage of their own tests, by getting flaky tests back to running or removing them.

You can leave any feedback about this process in the dedicated issue.

Goals

- Increase master stability to a solid 95% success rate without manual action
- Improve productivity - MR merge time - lower "Average Retry Count"
- Remove doubts on whether master is broken or not
- Reduce the need to retry a failing job by default
- Define acceptable thresholds for action like quarantining/focus on refactoring
- Step towards unlocking Merge train

Additional resources

- Flaky tests technical documentation
- Measure and act on flaky specs
- Flaky tests dashboard

SECTION: engineering/testing/guide-to-e2e-test-failure-issues.md

Troubleshooting Failure Issues (Video 3 minutes)

Most Common Fixes

- Element not found? · Check if UI changed in recent MRs
- Timing out? · Look for spinners in screenshot, check performance, check for page errors
- 401 Unauthorized? · Token expiration issue
- Only in staging-canary/staging environment? · Check staging channel for environmental issues and recent feature flag toggles

Debugging the Failure

1. Check the screenshot and exception for any obvious errors, examples:
   - ElementNotFound · UI element missing/changed

- TimeoutError · Unexpected behavior or slow loading
- AssertionError · Unexpected data or behavior
- WaitExceededError · Look for spinners still loading in screenshot
- 401 Unauthorized · Check expiring tokens
- Server errors displayed in the UI · environmental issues or test set-up issues

2. Check GitLab instance under test - Use the found: labels (note: failure can be across multiple instances)
- found:master · Ephemeral environment, failed in scheduled pipeline against master branch
  - Open the latest failed job from the Reports section
  - Check where the test is failing (GDK, CNG, Omnibus)
  - View the job name - this indicates the test configuration
  > Note: Merge Requests will be blocked when tests are failing against GDK and CNG. These failures tend to be flaky failures as the test would have usually failed in a previous merge request. Tests against Omnibus are optional and allowed to fail.
- found:<environment> · Failed in a live environment (debugging guide)
  - Open job to see if failure was a smoke job, or check metadata of test to see if test is a smoke test
  - If test is failing in a single environment, check environment status (staging, production)
  > Note: :smoke test failures in staging-canary will block deployments.

3. Check failure frequency and timing
- Observe when the failure issue created to identify the first occurrence
- Observe the frequency of occurrences in the Reports section
- Failure patterns:
  - Multiple recent consistent failures are more likely to be a real issue, needing immediate action
  - Sporadic failures could mean test flakiness OR application instability (race conditions, timing issues)
  - Use the first occurrence time to check commits/deployments immediately prior to the issue occurring

4. View the test file for recent changes
- Click the File URL link in the failure issue metadata and review recent commits to the test file

5. Try to reproduce locally against your GDK, example:

```
shell
  cd qa
  bundle install
  WEBDRIVERHEADLESS=false GITLABQAADMINACCESSTOKEN=<admin PAT> QALOGLEVEL=DE
QAGITLABURL=http://gdk.test bundle exec rspec
qa/specs/features/browserui/3create/repository/addfiletemplatespec.rb
```

6. If failure is from a live environment and passing against GDK, try against live environment or manually verify the functionality works in live environment.
- https://docs.gitlab.com/development/testingguide/endtoend/featureflagtesting/running-e2e-tests-against-staging

7. Check application logs for signs of failure
- Check job artifacts for master failures

   - Check https://nonprod-log.gitlab.net for staging failures
   - Check https://log.gprd.gitlab.net for production failures
8. Check subsequent test runs
   - Click Test case link in this issue
   - Check labels in test case issue for latest status of test case · If the test has subsequently passed the test or environment may be flaky
9. Check recent feature flag toggles (if failure is in a live environment)

 Triage Actions

Apply appropriate label per classification guide

> Note: Failure issues will be auto-closed after 30 days of no updates.

| Symptom
| Label | Action
|
|---|---|---|
| Feature broken, urgent (affects users)
| ~failure::bug | Create bug fix or revert MR
|
| Feature broken, non-urgent
| ~failure::bug | Create bug fix or quarantine + schedule fix for future milestone |
| Test stale/broken
| ~failure::stale-test | Update test or quarantine + schedule fix for future milestone
|
| Flaky test | ~failure::flaky-test | Investigate root cause + schedule fix for future milestone |
| One-off environment issue
| ~failure::test-environment | Monitor and close issue if does not re-occur
|
| External dependency failure
| ~failure::external-dependency | Monitor and close issue if does not re-occur
|

Flakiness can be caused by the test OR the application itself being unreliable under certain conditions

 Quarantining tests

Quarantine is a temporary measure for:

- Stale/broken tests (Feature works for users)
- Known acceptable issue causing :smoke test failures or excessive noise

1. Use Fast Quarantine for an urgent quarantine
2. Follow up with a long-term quarantine

3. Tag this issue with ~quarantine and ~automation:prevent-auto-close

Quarantining-tests full guide

Need further assistance?

Contact gtestgovernance Slack channel or create a Test Governance Request for help issue

---
SECTION: engineering/testing/observability_performance.md

Description

Observability Based Performance Testing is a proactive approach to understanding system performance through comprehensive instrumentation and real-time data collection. Unlike traditional performance testing, which relies on specific test scenarios, observability testing provides deep visibility into application behavior under real-world conditions.

Approach

Observability testing is actively making use of our Observability tools to detect trends that would develop into performance issues. A couple common approaches:

- Have the development teams monitor the dashboards on their components and proactively pickup performance concerns
- Build dashboards/tooling that support doing exploratory testing on the Observability data, looking for linkages that may not be obvious (system A causes system B to slow down)
  - Tools like the Performance Bar can enable someone to notice a performance oddity and start the investigation into the root cause
- Extend existing Observability tooling to development/test environments
  - This enables teams to get performance metrics earlier in the development process
  - This increases team familiarity with the tooling which facilitates easier adoption/use

Key Components

- Instrumentation: Embedding traces, metrics, and structured logging across the application stack.
- Real-time Data Collection: Gathering performance data continuously during actual usage.
- Holistic Analysis: Examining the entire system to identify bottlenecks and patterns.

Goals

- Enhanced Performance Visibility: Increase team awareness of their work's performance impact.
- Proactive Issue Detection: Identify potential problems before they affect customers.
- Team Empowerment: Provide accessible performance analysis tools that don't require specialized expertise.

Non-Goals

- Replace Load Testing: Observability-based testing complements, but does not replace, well-designed load tests like those using GPT.

Implementation Approach

Phase 1: Foundation Setting

1. Education & Documentation
   - Add documentation on what this process is so teams understand it
   - Create training to help teams understand and adopt
   - Create [guides(https://gitlab.com/gitlab-org/quality/quality-engineering/team-tasks/-/workitems/3323)] to enable teams to build / use performance dashboards
2. Baseline Assessment & Infrastructure
   - There are a number of existing dashboards, identify existing ones that already cover what we need
   - Identify gaps and develop plans to fill

Phase 2: Initial Implementation

1. Team Selection
   - Select a single team to pair with
   - Work with to identify / build dashboards for their use
   - Integrate with their workflow
2. Feedback Loop & Documentation
   - Bring lessons learned back into documentation for other teams

Phase 3: Practice and Culture Building

1. Expand to Early Adopters
2. Process Integration
   - Document how to use in GitLab Dev workflows

Phase 4: Expansion and Maturity

1. Full Scale Rollout
2. Continuous Improvement
3. Advanced Capabilities
   - Machine Learning Integration: Implement ML algorithms to predict performance issues based on observability data patterns.
   - Enhanced Developer Tools: Create IDE plugins that provide real-time performance insights during code writing.

Case Studies

Case Study 1: Scrum team using Observability Based Performance Testing

Background

The team was building an ETL (Extract, Transform, Load) project with critical performance requirements. A few hours of system delay would create an insurmountable backlog in the ETL

process, making performance monitoring essential to business operations. The team was small (9 developers, 2 QA, 1 Ops) and deployed to production every other week, adding a heavy weight process would of introduced a significant burden. So, we implemented using our existing Observability stack.

Observability implementation

We implemented a three-tiered monitoring approach:

1. User Experience Monitoring
   - Used existing Apdex-based dashboards for both production and test environments
   - Tracked end-user performance metrics like page load times and API response times
   - Set clear thresholds for acceptable performance levels
2. Service-Level Monitoring
   - Created detailed dashboards for key services showing:
     - Resource utilization (CPU, memory, I/O)
     - Service dependencies and their performance
     - Queue lengths and processing rates
   - Established KPI tracking for critical ETL operations
3. Error Tracking
   - Implemented comprehensive error logging
   - Created error pattern recognition dashboards
   - Set up alerting for error rate thresholds

Daily Workflow

1. Dashboard Review (5 minutes in daily standup)
   - Team reviewed all three dashboard tiers
   - Focused on identifying patterns rather than immediate problem-solving
   - Used a structured approach:
     - Review user experience metrics
     - Check service health indicators
     - Examine error patterns
2. Issue Management
   - Created issues for concerning patterns
   - Prioritized performance issues alongside feature work
   - Allocated dedicated investigation time in sprint planning

Implementation Challenges

- Small team size and fast release cycle was a constraint on what we could deliver
- Balancing prioritizing performance investigation with new feature development was difficult
  - Performance improvements often competed with feature deadlines
  - Sometimes had to make trade-offs between immediate fixes and long-term solutions
- It took some trial and error to identify the right metrics/dashboard configuration for our team
  - What we thought was the right metric to monitor proved not to be correct
  - The infrastructure evolved incrementally so we had to evolve the dashboards to keep in sync

Outcomes

- Met performance KPIs consistently
- Maintained 99.999% uptime during 10x growth period
  - Zero performance-related incidents
  - No emergency load testing needed
  - Proactively scaled based on observability data
- Lessons Learned:
  - Early warning from observability prevented major incidents
  - Daily review habit kept performance top-of-mind
  - Small, regular improvements are better than big reactive changes

Tools and Technologies

- Prometheus
- Grafana
- [Add other relevant tools used at GitLab]

Future Directions

[TBD]

Resources

- GitLab Handbook pages
  - GitLab Performance Monitoring
  - Observability for stage groups
- [Link to related blog posts or external resources]
- [Information on available training or workshops]

SECTION: engineering/testing/oncall-rotation.md

Developer Experience Sub-Department pipeline triage on-call rotation

This is a schedule to share the responsibility of debugging/analysing the failures
in the various scheduled pipelines that run on multiple environments.
Developer Experience sub-department's on-call does not include work outside GitLab's normal business hours. Weekends and Family and Friends Days are excluded as well. SETs also have full autonomy to move their 1:1s and their attendance to department meetings asynchronous for the week of their oncall if they decide to do so but please communicate to your manager accordingly.

In the current iteration, we have a timezone based rotation and triage activities happen during each team member's working hours.

Please refer to the Debugging Failing tests
guidelines for an exhaustive list of scheduled pipelines
and for specific instructions on how to do an appropriate level of investigation and determine next steps for the failing test.

Responsibility

Triage, record, and unblock failures

- The Saturday before the new rotation begins, the handover bot will create an issue in the pipeline-triage project.
- The handover bot also assigns the triage issue created to the DRIs for the upcoming week, according to the schedule below.
- During the scheduled dates, the support tasks related to the test runs become the Directly Responsible Individual's ([DRI]'s) highest priority.
- Reporting and analyzing the End to End test failures in Production, Canary, and Staging pipelines takes priority over GitLab master, and GitLab FOSS master. Note this also takes priority over fixing tests.
- Preprod pipelines have equal priority with Production and Staging pipelines during release candidate testing from the Monday through Thursday of the release week.
- If there is a time constraint, the DRI should report and analyze the failures in Staging, Canary and Production pipelines just enough to determine if it is an application or an infrastructure problem, and escalate as appropriate. All the reported failures in those pipelines should be treated as ~priority::1/~severity::1 until it's determined that they're not. That means they should be investigated ASAP, ideally within 2 hours of the report. If the DRI will not be able to do so, they should delegate any investigation they're unable to complete to the DRI from the following week. If neither DRI is available or will be able to complete the investigations, solicit help in the sdeveloperexperience slack channel.
- Consider blocking the release by creating an incident if new ~severity::1 regressions are found in smoke specs.
- It is important that all other failure investigations are completed in a timely manner, ideally within 24 hours of the report. If the DRI is unable to investigate all the reported failures on all the pipelines on time, they should solicit help in the sdeveloperexperience slack channel.
- Cross-cutting issues such as https://gitlab.com/gitlab-org/quality/team-tasks/-/issues/530 are triaged by the on call DRI to determine the next action. Other team-members should notify the DRI if they come across such an issue. The DRI can inform the rest of the department via the sdeveloperexperience channel, if necessary.
- Everyone in the Developer Experience sub-department should support the on-call DRI and be available to jump on a zoom call or offer help if needed.
  - If a change needs to be made to CI/CD variables in gitlab-org/gitlab and the DRI does not have access, they can ask for help in the dxmaintainers and development Slack channels. Any Quality maintainer should have sufficient access to be able to assist.
- Every day the APAC/EMEA/AMER DRI summarizes the failures found for that day in the triage issue like this. This serves as an asynchronous handoff to the DRI in the other timezone.
- The DRI can make use of the dri gem to automate reporting handoff and surface useful triage information, such as newly created failures or feature flags enabled by environment.
- Be aware that failing smoke specs in staging/canary will block the deployer pipeline and may be treated as production incidents.

Schedule

To view the pipeline triage rotation schedule, please visit the pipeline-triage project.

Or use chatops in Slack with command /chatops run quality dri schedule

Responsibilities of the DRI for scheduled pipelines

- The DRI does the triage. They should solicit help in the sdeveloperexperience slack channel if they need further assistance.
- The DRI makes the call whether to fix or quarantine the test.
- The DRI reviews any automated quarantine MRs and automated dequarantine MRs and decides whether to merge or close.
- The fix/quarantine MR should be reviewed by the counterpart SET. If the counterpart SET is not available immediately or if there is no counterpart SET, then any other SET can review the MR. In any case, the counterpart SET is always CC-ed in all communications.
- The DRI should periodically take a look at the list of unassigned quarantined issues and work on them.
- If the DRI is not available during their scheduled dates (for more than 2 days), they can swap their schedule with another team member. If the DRI's unavailability during schedule is less than 2 days, the DRI in the same timezone the following week can cover.

Responsibilities of the DRI for deployment pipelines

- The DRI helps the delivery team debug test failures that are affecting the release process.
- The DRI in the same timezone for the following week fills in when the DRI is not available.

New hire on-call shadowing

To help a new hire prepare for their first on-call rotation, they should spend a few days before their first rotation shadowing the DRI. When the new hire is DRI, the DRI from the following week should pair with the new hire for a couple of days to answer questions and help triage/debug test failures.

Supporting infrastructure and environment upgrades

During planned upgrades to live environments during GitLab's regular business hours, the Developer Experience sub-department may be requested to validate the environment pre/post-upgrade by running our end-to-end test suite. The DRI shall take responsibility for any assistance requested in triggering or reviewing test results.
This should be planned at least 1 week in advance, including the relevant DRIs QEM and SET who will be on call on the proposed date.
NB - For assistance with supporting upgrades outside GitLab's regular business hours, please submit a RFH (request for help) issue.

Some examples of issues where Developer Experience have provided support include:

- Sec Decomposition GPRD Rollout
- CI Decomposition Rollout
- PostgreSQL 14 upgrade

If the assigned DRI is unavailable during the planned upgrade, then the assigned DRI is to reach out to find coverage for the day of the planned upgrade.

- For example, if the planned upgrade is scheduled for outside of working hours (i.e., on a Saturday) and the assigned DRI cannot be made available, the assigned DRI is to reach out to

the team to find another team member who can be present throughout the planned upgrade.

Developer Experience Sub-Department incident management on-call rotation

The EMs should share the responsibility of monitoring, responding to, and mitigating incidents. Developer Experience Sub-Department's on-call does not include work outside GitLab's normal business hours. Weekends and Family and Friends Days are excluded as well.
In the current iteration, incident management activities happen during each team member's working hours.

Responsibility

- The Engineering Manager should ensure they have joined the Slack channel incidents.
- The Engineering Manager should help with monitoring the incident management channel, tracking, directly helping, delegating, and raising awareness of incidents within the Developer Experience Sub-Department as appropriate.
- The current DRI should be clearly noted on the incident issue.
- If a corrective action is needed, the EM should create an issue and ensure it is labeled with ~'corrective action'.
- Everyone in the Developer Experience Sub-Department should support the on-call DRI and be available to jump on a Zoom call or offer help if needed.

SECTION: engineering/testing/operational-verification.md

Overview

Between Self-managed, Dedicated and SaaS, we are going to have a large number of GitLab instances (Tenants and Cells) running that we will need to deploy to. Every time we deploy, we need to verify that the deploy was successful and the instance came back up correctly. We could verify the deploy through the use of E2E (End to End) test scripts. However this is a very slow, expensive, and invasive approach. E2E tests work through the UI/API of GitLab which makes them vulnerable to changes in functionality (which are frequent for an application that is still under active development), often require administrator access, and generate data on the instance we are testing. All three of these are not very desirable on customer facing instances.

Many of the checks that we need to do would be painful or difficult to test and report on from an E2E perspective. For example, verifying that the GitLab instance is able to connect to all it's data stores correctly could only be tested indirectly in an E2E scenario, by finding that some functionality does not work, but did it not work becuase the data store was not accessible or because some feature flag was not turned on or ...?

A better approach would be to enhance GitLab's endpoints / health checks to allow the instance to tell us that it is configured correctly and getting responses, that way we are getting direct information on the check.

mermaid
flowchart TB

```
subgraph Goal["Performance Testing Enablement"]
   Portal["Developer Portal (Self-Service)"]
end

subgraph P1["Pillar 1: Deployment Verification"]
   DV1["Instance Stability"]
   DV2["Reliability Checks"]
end

subgraph P2["Pillar 2: Feature Performance"]
   subgraph Load["Load Testing"]
      L1["GPT Tests"]
      L2["Reference Architecture"]
   end
   subgraph Dev["Developer Focus"]
      D1["Instrumented Testing"]
      D2["Contract Testing"]
      D3["Profiling"]
   end
   subg
```